Zadanie 04 - Room Booking API: authentication, authorization i konfiguracja

Założenia

Rozwijamy Room Booking API z zadania 03. Dodajemy użytkowników, logowanie i kontrolę dostępu.

Zmienia się model rezerwacji: pole tekstowe `reservedBy` zostaje zastąpione przez `userId` - powiązanie z kontem użytkownika, który dokonał rezerwacji.

Role w systemie:

Rola	Może
`USER`	przeglądać budynki i sale, tworzyć rezerwacje, przeglądać i anulować **własne** rezerwacje
`ADMIN`	wszystko co `USER` + zarządzać budynkami i salami, przeglądać i anulować **wszystkie** rezerwacje

Wymagane technologie: jak w zadaniu 03 + `bcryptjs` (lub `bcrypt`), `jsonwebtoken`, `swagger-ui-express`.

Za cały zestaw można uzyskać maksymalnie **5 punktów**.

Każde zadanie jest warte **1 punkt**.

Zadanie 1 - Rejestracja - 1 pkt

Utwórz endpoint:

```
```

```
POST /api/auth/register
```

Przykładowy request:

```
{
  "email": "Jan.Kowalski@example.com",
  "password": "bardzo-dlugie-haslo"
}
```

Przykładowy response `201 Created`:

```
{
  "id": 2,
  "email": "jan.kowalski@example.com",
  "role": "USER",
  "createdAt": "2026-11-03T09:00:00.000Z"
}
```

Wymagania

- tabela `users` utworzona **nową** migracją (`id`, `email` z `UNIQUE`, `password_hash`, `role` z `CHECK`, `created_at`),
- email walidowany i **normalizowany** do małych liter,
- hasło: minimum 10, maksimum 72 znaki,
- hasło zapisane jako hash bcrypt (cost factor minimum 10) - w bazie nie ma hasła w żadnej odwracalnej postaci,
- nowy użytkownik **zawsze** dostaje rolę `USER`; próba przesłania pola `role` (lub dowolnego innego nieznanego pola) → `400`,
- zajęty email → `409` z kodem `EMAIL_TAKEN`; również przy **równoległych** rejestracjach tego samego emaila (constraint `UNIQUE` + mapowanie `23505`, żadnego `500`),
- response **nie zawiera** `password_hash` / `passwordHash` w żadnej formie,
- konto ADMIN tworzone w seedzie (nie da się go utworzyć przez API).

W sprawozdaniu pokaż wiersz z tabeli `users` (z widocznym hashem) oraz próby: rejestracji z rolą `ADMIN`, zbyt krótkiego hasła, duplikatu emaila różniącego się wielkością liter.

Zadanie 2 - Logowanie i JWT - 1 pkt

Utwórz endpointy:

```
POST /api/auth/login
GET  /api/auth/me
```

Response logowania:

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "tokenType": "Bearer",
  "expiresIn": "15m"
}
```

Wymagania

- token JWT podpisany algorytmem `HS256`, z claimami `sub` (ID użytkownika jako string), `role`, `iat`, `exp`,
- czas życia tokenu konfigurowalny (zmienna środowiskowa), domyślnie 15 minut,
- payload **nie zawiera** emaila, hasha ani innych danych osobowych,
- zły email i złe hasło dają **identyczną** odpowiedź `401` (ten sam kod, ten sam komunikat),
- `GET /api/auth/me` zwraca dane zalogowanego użytkownika (bez hasha),
- weryfikacja tokenu z jawnie określonym algorytmem (`algorithms: ["HS256"]`).

W sprawozdaniu pokaż zdekodowany payload tokenu (np. z jwt.io) oraz porównanie odpowiedzi dla złego emaila i złego hasła.

Zadanie 3 - Middleware authenticate i rezerwacje użytkownika - 1 pkt

Wymagania

- middleware `authenticate` odczytuje token z nagłówka `Authorization: Bearer <token>` i ustawia `req.user = { id, role }`,
- poprawna obsługa przypadków (wszystkie → `401`; wygasły token z osobnym komunikatem, np. `Token expired`, żeby frontend wiedział, że wystarczy ponowne logowanie):
  - brak nagłówka,
  - nagłówek w złym formacie (np. `Basic ...`, `Bearer` bez tokenu),
  - token ze złym podpisem (zmień jeden znak w tokenie),
  - token zmodyfikowany (zmieniony payload, np. `role` na `ADMIN`, z oryginalnym podpisem),
  - token wygasły (do testu ustaw tymczasowo `JWT_EXPIRES_IN=10s`),
- **wszystkie** endpointy poza `/api/auth/register`, `/api/auth/login` i `/api/docs` wymagają uwierzytelnienia,
- **nowa** migracja: kolumna `reservations.user_id` (FK do `users`) zastępuje `reserved_by`; istniejące rezerwacje z seeda muszą zostać poprawnie przypisane do użytkowników (napisz migrację tak, żeby działała na bazie z danymi, a nie tylko na pustej),
- `POST /api/reservations` ustawia `userId` **wyłącznie** na podstawie tokenu; pole `userId` / `reservedBy` w body → `400`,
- response rezerwacji zawiera `user: { id, email }` zamiast `reservedBy`.

W sprawozdaniu pokaż wszystkie przypadki `401` oraz kod migracji z `user_id`.

Zadanie 4 - Authorization: role i ownership - 1 pkt

Wymagania

Role:

- middleware `authorize(...roles)`,
- `POST`, `PATCH`, `DELETE` na `/api/buildings` i `/api/rooms` - tylko `ADMIN` (`USER` → `403`),
- `GET` na budynkach i salach - każdy zalogowany użytkownik.

Ownership:

- `GET /api/reservations`:
  - `USER` widzi **tylko swoje** rezerwacje - filtr dokłada backend, niezależnie od parametrów query,
  - `ADMIN` widzi wszystkie; dodatkowy filtr `?userId=` działa **tylko** dla ADMINa (dla USERa → `403` albo ignorowany - wybierz i uzasadnij),
  - pagination i pozostałe filtry z zadania 03 nadal działają,
- `GET /api/reservations/:id` i `PATCH /api/reservations/:id/cancel`:
  - właściciel albo `ADMIN` → normalne działanie,
  - inny `USER` → `404` (rezerwacje są prywatne - nie zdradzamy, że istnieją),
- sprawdzenie ownership znajduje się w **service** (który dostaje `currentUser` jako parametr), a nie w controllerze,
- reguły biznesowe z poprzednich zestawów (kolizje, pojemność, anulowanie rozpoczętej rezerwacji) działają również dla ADMINa.

W sprawozdaniu przygotuj tabelę: endpoint × (brak tokenu / USER właściciel / USER obcy / ADMIN) → status code, i potwierdź każdy wiersz screenshotem lub logiem z pliku `.http`.

Zadanie 5 - Konfiguracja, sekrety i dokumentacja API - 1 pkt

Część A - konfiguracja

- jeden moduł konfiguracji (`src/config`), który waliduje zmienne środowiskowe schematem Zod,
- brakująca lub niepoprawna zmienna (np. `JWT_SECRET` krótszy niż 32 znaki, niepoprawny `DATABASE_URL`) → aplikacja **nie startuje** i wypisuje czytelny komunikat,
- **żaden** inny plik nie czyta `process.env` bezpośrednio (również `knexfile.js`),
- `.env.example` w repozytorium, `.env` w `.gitignore`,
- w repozytorium (w **całej historii** git) nie ma prawdziwych sekretów. Sprawdź to poleceniem:

```
git log -p --all | grep -iE "JWT_SECRET=|password="
```

Jeżeli sekret kiedykolwiek trafił do historii - wygeneruj nowy i opisz to w sprawozdaniu.

Część B - dokumentacja

- Swagger UI dostępny pod `/api/docs`,
- udokumentowane **wszystkie** endpointy, z parametrami, schematami body i **wszystkimi** możliwymi kodami odpowiedzi (`2xx` i `4xx`),
- `securitySchemes` typu `bearer` - przycisk **Authorize** działa i pozwala wywołać chronione endpointy z poziomu Swagger UI,
- schemat formatu błędu (`{ error: { code, message, details, requestId } }`) zdefiniowany raz w `components/schemas` i używany przez referencję (`$ref`).

W sprawozdaniu pokaż: start aplikacji z błędną konfiguracją, ekran Swagger UI z listą endpointów oraz udane wywołanie chronionego endpointu przez Swagger UI po autoryzacji.

Punktacja

Zadanie	Punkty
Rejestracja	1 pkt
Logowanie i JWT	1 pkt
Middleware authenticate i rezerwacje użytkownika	1 pkt
Authorization: role i ownership	1 pkt
Konfiguracja, sekrety i dokumentacja API	1 pkt
Razem	**5 pkt**

Co oddać?

Przygotuj sprawozdanie w formacie PDF.

Powinno ono zawierać:

- fragmenty najważniejszego kodu (auth service, middleware `authenticate` i `authorize`, sprawdzenie ownership w service, moduł konfiguracji, migracje),
- screenshoty potwierdzające działanie endpointów,
- tabelę uprawnień z zadania 4,
- przykłady obsłużonych błędów: `400`, `401`, `403`, `404`, `409`,
- krótki opis rozwiązania i uzasadnienia decyzji, o które prosi treść zadań.

Nie ma potrzeby umieszczania screenshotu całego kodu.

Najważniejsze jest pokazanie, że API działa zgodnie z wymaganiami.

Link do repozytorium GitHub - opcjonalnie (zalecany).